
Yin-Yang et TouIST : Modélisation et résolution de contraintes pour jeux de plateau

Résumé

Ce rapport présente la modélisation du jeu Yin-Yang avec l'outil TouIST. Il décrit les choix de représentation des états de la grille, les encodages des contraintes (R1, R2, R3), les optimisations mises en œuvre pour des grilles 6×6 et 10×10 , ainsi que les principales difficultés rencontrées et les solutions adoptées.

Table des matières

Introduction

Le jeu Yin-Yang se joue sur une grille $N \times N$ où certaines cases sont préremplies par des pions blancs (B) ou noirs (N). Les règles à modéliser sont :

- (R1) interdiction des carrés 2×2 monochromes ;
- (R2) connexité unique des cellules noires ;
- (R3) connexité unique des cellules blanches.

L'objectif du projet est de produire des fichiers TouIST qui codent ces contraintes et permettent de résoudre les grilles fournies (grilles 1 à 4 du sujet).

Méthode

Représentation. Nous utilisons deux familles d'encodage :

- Encodage A (variables explicites) : $B(i, j)$ et $N(i, j)$ pour chaque case (i, j) . On impose l'exclusivité (une case est soit blanche soit noire) et on encode (R1) par des clauses interdisant chaque configuration 2×2 uniforme.
- Encodage B (optimisé) : une seule variable $N(i, j)$ (vrai si noir, sinon blanc) ou, pour la connexité, des variables $r(e, c, i, j)$ signifiant que la case (i, j) appartient au territoire de couleur c à l'étape e (construction inductive du territoire).

Connexité. Pour (R2) et (R3) nous construisons un territoire en L étapes ($e = 0 \dots L$, avec $L \leq N \times N$) :

- étape 0 : exactement une des quatre cases du coin supérieur gauche appartient au territoire de couleur c ;
- propagation : si une case contenant un pion de couleur c est adjacente (par côté) à une case dans le territoire à l'étape $e-1$, elle peut entrer au territoire à l'étape e ;
- monotonie : une fois dans le territoire, une case y reste.

Optimisations. Pour réduire la taille du modèle et accélérer le solveur :

- n'utiliser que $N(i, j)$ et déduire la couleur par négation quand c'est possible ;
- encodage par territoires pour éviter des clauses globales coûteuses ;
- briser les symétries en fixant des conventions sur le coin initial et en minimisant L .

Résultats

Fichiers fournis : exercice1.touist, exercice2.touist, grille1.touist, grille2.touist, grille3.touist, grille4.touist (optimisé). Ces fichiers implémentent les encodages décrits ci-dessus.

Les modèles obtenus pour les grilles 6×6 correspondent aux solutions attendues dans le sujet (Figures 4–6). Pour la grille 10×10, l’encodage optimisé permet d’obtenir la solution de la Figure 8 en ajustant la borne L à la valeur minimale nécessaire.

Difficultés rencontrées et solutions

- Connexité : coder proprement la propagation par étapes nécessite plusieurs formules distinctes selon la géométrie locale (coins, bords, intérieur). Solution : écrire des clauses séparées pour chaque cas et factoriser par fonctions de génération de clauses.
- Explosion combinatoire (2×2 et exclusivité) : la génération brute des clauses 2×2 peut alourdir le solveur. Solution : passer à l’encodage optimisé et réduire les variables redondantes.
- Choix de L : trop grand, le modèle devient lent ; trop petit, pas de solution. Solution : tester L par incréments et observer la plus petite valeur donnant une solution.

Difficultés induit par la syntaxe touist

1. Compréhension du fonctionnement des comparateurs $>$, $<$, $\leq$, $\Rightarrow$, $\neq$

Erreur

```
bigand $c, $e, $i, $j in $C, [1, ($L)], [1, ($N)], [1, ($N)]:
  (r($e,$c,$i,$j) and not r($e-1,$c,$i,$j)) =>
  (
    (($i >= 2) then r($e-1,$c,$i-1,$j) else Bot end) or
    (($i < $N) then r($e-1,$c,$i+1,$j) else Bot end) or
    (($j >= 2) then r($e-1,$c,$i,$j-1) else Bot end) or
    (($j < $N) then r($e-1,$c,$i,$j+1) else Bot end)
  )
end
```

Correction

```
$c, $e, $i, $j in $C, [1, ($L)], [1, ($N)], [1, ($N)]:
  (r($e,$c,$i,$j) and not r($e-1,$c,$i,$j)) =>
  (
    (if ($i >= 2) then r($e-1,$c,$i-1,$j) else Bot end) or
    (if ($i < $N) then r($e-1,$c,$i+1,$j) else Bot end) or
    (if ($j >= 2) then r($e-1,$c,$i,$j-1) else Bot end) or
    (if ($j < $N) then r($e-1,$c,$i,$j+1) else Bot end)
  )
end
```

2. Intuition sur les end en fin

Erreur

```
bigand $i, $j in [1, ($N)], [1, ($N)]:
  (B($i,$j) xor N($i,$j))
```

Correction

```
bigand $i, $j in [1, ($N)], [1, ($N)]:  
    (B($i,$j) xor N($i,$j))  
end
```

Pourquoi ces choix

Les encodages proposés équilibrent clarté et performance. L'encodage explicite est facile à vérifier et pédagogique (utile pour `exercice1/exercice2`), tandis que l'encodage par territoires est pragmatique pour résoudre efficacement des instances plus grandes.

Utilisation d'IA générative

Nous n'avons pas eu recours à une IA générative pour écrire les modèles TouIST. Car chatgpt disait n'importe quoi !

Conclusion

Le projet montre comment formaliser des contraintes de jeu (R1–R3) en logique propositionnelle et comment optimiser un encodage pour la rendre utilisable sur des grilles non triviales. Travaux futurs : automatiser la génération de clauses en FNC, tester d'autres heuristiques de résolution et documenter les performances sur des instances plus grandes.